UNMASK: Evidence-Bounded Comparison of Published npm Packages Against Their Source Repositories for Supply-Chain Review

Pranay Vishwakarma
Team Leader

Vaiditya Tanwar
Analyzer, Model & CI Lead

Vishesh Rao
CLI & Release Setup

Sayan Bhattacharya
Documentation & Report

Department of Computer Science and Artificial Intelligence (CSAI)
Newton School of Technology, Rishihood University, Sonipat, India
Hacktoberfest 2026, Hack Day Sonipat — Team Cappybara Project Report

***Abstract*—A published software package is a separate artifact from the repository that claims to produce it. Nothing in the npm registry forces the two to agree, and several high-profile attacks have shipped code in the tarball that never appeared in reviewed source. This report describes UNMASK, a package-integrity analyzer built by Team Cappybara for Problem 04 of Hacktoberfest 2026 (Hack Day Sonipat): *Published Package vs Source Code Integrity Analyzer*. UNMASK inventories an npm tarball and a source archive pinned to a full commit SHA without executing either, classifies every regular file of the artifact into one of four correspondence states (byte-identical, changed, artifact-only, not-compared), binds each observation to a byte range in a digest-addressed blob, and derives a deterministic exit code from a small, versioned policy. An optional large language model (Google Gemma) may explain the evidence, but its output is advisory, must quote supplied evidence verbatim, and can never clear a deterministic finding. The model stage uses an ordered three-tier route (Gemini 3.5 Flash, Gemini 3 Flash preview, Gemma 4 26B) with bounded retries and strict output validation. We present the threat model, the separated-facts design principle, the archive and correspondence algorithms, the hosted process model and the release pipeline that binds a source commit, an npm tarball and a container image digest. The analyzer core, hosted CLI, HTTP server, Next.js website and 388 test declarations across 33 files are now merged, and version 0.1.0 is deployed on Google Cloud Run. Hosted model generation, previously blocked by an HTTP 403 quota error, now succeeds at the provider (HTTP 200). In three live spot checks the deterministic core completed every time; the model was correctly skipped for two fully matching packages, and for the third the validator rejected the model's output rather than show an unchecked explanation. We close with a measurable evaluation plan.**

***Index Terms*—software supply chain, npm, package integrity, reproducible builds, provenance, static analysis, large language models, evidence grounding, CI/CD**

I. Introduction

DEVELOPERS install packages, not repositories. When a team reviews a dependency it usually reads the GitHub source, yet what actually runs is the tarball downloaded from the registry. The two are linked only by convention: the publisher runs `npm publish` on a machine the reviewer never sees, possibly after a build, minification or a manual edit. Two properties make this gap dangerous. First, it is invisible by default: `npm install` verifies that the tarball matches the registry's recorded hash, not that it matches any source. Second, it has been exploited repeatedly, from the `event-stream` incident, where a malicious payload lived only in the published build of a dependency [1], to the `xz-utils` backdoor, whose trigger was present in release tarballs but not in the corresponding Git tree [2].

Existing responses fall into two families. *Attestation* approaches such as Sigstore [3], in-toto [4], SLSA [5] and npm provenance [6] sign statements about how an artifact was built; they are strong when adopted but say nothing about the large majority of packages that publish no attestation. *Reproduction* approaches rebuild the artifact from source and compare [7]; they are definitive when they succeed but require executing untrusted build scripts and a faithful toolchain. A reviewer facing an arbitrary package today needs a third option: an inert, bounded, explainable comparison that states exactly what is known and what is not.

This project addresses three design questions under a single, documented contract:

- **DQ1** Can the difference between a published npm tarball and a pinned source archive be computed *without executing* either, under explicit resource bounds, and still be useful to a reviewer?
- **DQ2** How should the result separate facts of different strength (registry integrity, source claim, byte correspondence, syntax cues, model hypotheses) so that no weaker fact silently upgrades a stronger one?
- **DQ3** Under what constraints can a hosted LLM add explanation value without becoming a trusted classifier?

The contributions of this phase are: (i) a threat model and a *separated-facts* principle that governs every report field; (ii) a specified, bounded archive inventory and a four-state correspondence classification with byte-range evidence; (iii) a deterministic, versioned exit policy that is independent of the model; (iv) an advisory model contract with mandatory verbatim citations; (v) a CI and release pipeline that binds

source commit, npm tarball and image digest; and (vi) a working, deployed implementation with live spot checks, an honest status audit and a measurable plan for evaluation.

The remainder of the report is organised as follows. Section II reviews related work. Section III states the problem and threat model, and Section IV the requirements and design decisions. Section V describes the architecture, Section VI the core methods, and Section VII the engineering and evaluation design. Section VIII reports the current status, Section IX discusses the design and alternatives, and Section X sets out limitations and future work.

II. Literature Review

A. Attacks on Package Ecosystems

Zimmermann et al. [8] showed that the npm dependency graph is a "small world": a handful of maintainer accounts can reach a large fraction of all packages, so a single compromised publisher has outsized reach. Ohm et al. [9] catalogued 174 malicious packages and found that most injected code runs at install time through lifecycle hooks or on first import. Duan et al. [10] built a measurement pipeline across npm, PyPI and RubyGems and confirmed hundreds of previously unknown malicious packages. These studies motivate two UNMASK choices: install hooks are first-class policy findings, and no target code is ever executed.

B. Source-to-Package Discrepancy

The work closest to ours is LastPyMile [11], which compared PyPI artifacts with their GitHub repositories and found that discrepancies are common and mostly benign (generated files, packaging metadata), while malicious injections hide among them. Vu et al. therefore argue that a useful tool must *explain* differences, not merely list them. UNMASK adopts this framing for npm and adds explicit byte-level evidence and a model-independent policy.

C. Provenance and Attestation

in-toto [4] records signed layouts of supply-chain steps; Sigstore [3] removes long-lived signing keys through short-lived certificates and a transparency log; SLSA [5] grades build integrity in levels. npm now records Sigstore-backed provenance for packages published from supported CI systems [6]. UNMASK records such claims as *labels* and states explicitly when they were not cryptographically verified.

D. Reproducible Builds

Lamb and Zacchiroli [7] describe reproducible builds as the strongest practical guarantee that a binary corresponds to its source. Their cost is a faithful, deterministic toolchain and the execution of build scripts. UNMASK deliberately stops short of rebuilding and makes no claim of build equivalence; it measures byte correspondence only.

TABLE I
Literature used and its role in this project

Strand	Refs.	Use in this project
Ecosystem attacks	[1], [2], [8], [9], [10]	Threat model; lifecycle findings; never execute
Source–package gap	[11]	Correspondence states; explain, not only list
Attestation	[4], [3], [5], [6]	Source-claim labels; unverified vs. verified
Reproducible builds	[7]	Explicit non-goal; no equivalence claim
LLM grounding	[12], [13], [14]	Advisory model; mandatory citations
Engineering	[15], [16], [17]	Runtime, archive parser, syntax parser

E. LLMs for Security Explanation

Large language models can summarise code and suggest hypotheses, but they hallucinate, including invented code and citations [12]. Retrieval and citation grounding reduce, but do not remove, this risk [13]. UNMASK routes explanation requests to Google's Gemini Flash models and falls back to the open-weight Gemma family [14]. UNMASK therefore restricts the model to bounded evidence and validates that every quotation exists at its claimed location.

F. Research Gap

Attestation covers packages that opt in; reproduction covers packages whose builds can be replayed; discrepancy studies measure ecosystems in aggregate. Few tools give an individual reviewer a bounded, non-executing, per-file comparison whose every claim is tied to evidence and whose verdict does not depend on an LLM. Table I summarises how each strand is used.

III. Problem Statement and Threat Model

A. Problem

Given an exact npm release `name@version` (or a local tarball), and a source archive at a full 40-character commit SHA, report for every file shipped in the tarball whether identical bytes exist in the selected source, localise any difference, flag install-time behaviour not explained by source, and present the result as evidence a human can verify. The tool must never declare a package "safe".

B. Threat Model

Adversary. A publisher (or an attacker holding publisher credentials) who can upload any tarball and write any metadata, including a misleading `gitHead`, a `repository` URL or a crafted file name. *Inputs are hostile.* Archives may contain path traversal (`../`), absolute paths, NUL bytes, symlinks and hard links, Unicode or case collisions, decompression bombs and pathological JavaScript. *Out of scope.* Compromise of the registry's own integrity hash, of GitHub's archive service, or of the host running UNMASK; and semantic analysis of whether a changed file is malicious.

TABLE II
FACTS RECORDED BY UNMASK AND WHAT EACH DOES *NOT* ESTABLISH

Fact	Established by	Does not establish
Registry integrity	SHA-512 of captured tarball	Any relation to source
Source claim	`gitHead`, user SHA, provenance field	Build origin; signature validity
Byte correspondence	Equal size and SHA-256	Semantic equivalence of changed files
Syntax cues	Bounded Acorn parse	Malice or safety
Model hypothesis	Validated quotation	Truth of the reasoning

TABLE III
REQUIREMENTS AND THE DESIGN DECISIONS THEY DROVE

Requirement	Decision
Hostile archives	Stream parse with `tar-stream`; app-owned path, type and collision checks
No execution	No install, no build, links never followed
Exact identity	Reject tags/ranges; full 40-char SHA; original bytes kept
Bounded resources	Per-stage budgets; incomplete ⇒ exit 2
Explainability	Evidence IDs bound to blob digest, side, path, byte range
Model safety	Advisory only; citations validated; credentials not in child
Same meaning everywhere	One core and one renderer for CLI and web
Release integrity	One tarball; image built from those exact bytes; digest pinned

C. Separated Facts

The central design rule is that facts of different strength are recorded separately and never merged (Table II). A registry integrity match does not imply a source match; a source claim does not imply build origin; a syntax cue does not clear a change; and a model hypothesis does not change any of the above.

IV. Requirements and Design Decisions

Each requirement below produced a concrete decision, summarised in Table III.

Inert analysis. Target code, install hooks and build scripts are never run, and archive links are inventoried but never dereferenced. This rules out rebuild-based verification in this phase.

Exact identity. Tags, ranges and branch names are mutable and are rejected as input; only `name@version` and full commit SHAs are accepted, and original compressed bytes are authoritative.

Bounded work. Every stage has a byte, count or time budget (Table V); exceeding one yields an *incomplete* result, never a silently truncated "pass".

Deterministic verdict. The exit code is a pure function of the deterministic report and a versioned policy; the model is excluded from it.

Shared core. The CLI and the hosted website call the same pipeline and the same escaped HTML renderer, so a report means the same thing on both surfaces.

TABLE IV
MODULE INTERFACES OF THE ANALYSIS CORE

Module	Responsibility
`acquire`	Exact spec parsing; bounded fetch from approved hosts, no redirects
`resolve`	Source candidates, trust labels, conflicting claims
`archive`	`readArchive`: inert, bounded inventory
`source`	Select an explicit or unambiguous package directory
`compare`	Correspondence states, coverage, evidence
`syntax`, `manifests`	Bounded JavaScript cues; lifecycle scripts
`policy`, `report`	Deterministic exit code; validated JSON
`model/client`	Evidence bundling; citation checks; advisory output
`server/runner`	Child lifetime, deadlines, output caps, cleanup
`viewer/render`	Shared escaped HTML evidence view

Source: `docs/contracts.md`, `docs/architecture.md`.

V. System Architecture

A. Components

The implementation is plain ESM JavaScript on Node.js 24.22.2 with two production dependencies, `tar-stream` [16] and `acorn` [17]. Table IV lists the core modules, now merged under `src/`. Two surfaces sit on top: a CLI whose public-package mode sends only the exact spec to the hosted service, with a separate `--offline` mode for local archives, and a Next.js 16 website (`web/`) that forwards `/check` to the analysis server and renders the report viewer.

B. End-to-End Pipeline

Fig. 1 shows the flow. Registry orchestration captures input bytes and then calls the same local pipeline that offline CLI mode uses, so online and offline results are comparable.

C. Process Boundaries

The CLI runs the pipeline in-process. The hosted service separates concerns across processes: the main process owns HTTP, the queue, the store and the model client; a trusted Node child runs the deterministic stage with a 512 MiB V8 old-space limit, a minimal environment and a 120 s deadline. Provider credentials stay in the main process and are never inherited by the child. Job directories have random names and kernel-held ownership leases, so a restart preserves live jobs and only removes directories it can prove are abandoned.

D. Hosted Job Lifecycle

A visitor submits `name@version` via `POST /check`; the server returns `303` to a private `/r/<32-hex-id>` location. Jobs move through *queued* → *running* → {*done*, *incomplete*, *failed*} and then become unavailable after TTL, eviction or restart. The queue holds one active and one waiting job; admission is limited to six new checks per ten minutes; GET requests never start or retry work.

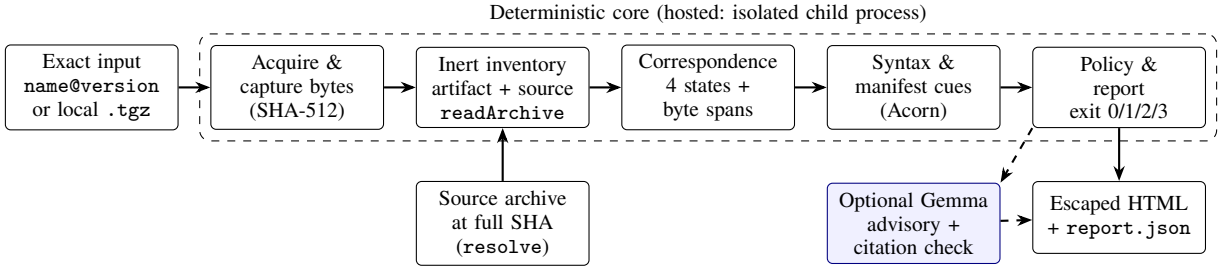


Fig. 1. End-to-end pipeline. The deterministic core fixes the exit code before the optional model stage (dashed) runs; the model can annotate but never alter it. Source: `docs/architecture.md`.

VI. Methodology

A. Problem Formulation

Let $A = \{a_1, \ldots, a_n\}$ be the regular-file entries of the artifact after mapping (one validated leading path component stripped), and S the regular entries of the selected source package directory. Each entry e has a normalised path $p(e)$, size $|e|$ and digest $h(e) = \mathrm{SHA256}(e)$. Let $\kappa(S) \in \{0, 1\}$ indicate that the source inventory is *complete* and *unambiguous*, and let $L(p)$ indicate that a source link exists at p or any ancestor of p.

B. Archive Inventory

`readArchive` streams gzip-compressed tar bytes and records, for each entry, its original and effective path, type, size, SHA-256 and inspection status. It rejects unsafe paths, unsupported types, traversal, NULs and canonical collisions, including case-folded and Unicode-normalised collisions:

$$\exists\, e \neq e' : \mathrm{NFC}(\mathrm{fold}(p(e))) = \mathrm{NFC}(\mathrm{fold}(p(e'))) \Rightarrow \text{reject}. \tag{1}$$

Content is retained up to 10 MiB per file within a shared 128 MiB budget; larger files are stream-hashed so that no misleading prefix is ever compared.

C. Correspondence Classification

For each $a \in A$ with counterpart $s \in S$ at the same path, the state is

$$\sigma(a) = \begin{cases} \textsf{byte-identical} & s \text{ exists},\ |s| = |a|,\ h(s) = h(a) \\ \textsf{changed} & s \text{ exists and usable},\ h(s) \neq h(a) \\ \textsf{artifact-only} & s \text{ absent},\ \kappa(S) = 1,\ \neg L(p(a)) \\ \textsf{not-compared} & \text{otherwise.} \end{cases} \tag{2}$$

The rule in (2) is deliberately conservative: absence is asserted only when the inventory is complete and no link could hide the file, so an incomplete source can never manufacture an artifact-only finding. Every artifact file appears exactly once, so

$$\textstyle\sum_\sigma N_\sigma = |A|, \tag{3}$$

which the report validator checks.

D. Difference Localisation and Evidence

For a changed file with both sides retained, the first and last differing byte offsets i and j define a half-open span $[i, j)$ on each side. An evidence record is the tuple

$$\varepsilon = (id,\ \mathrm{SHA256(blob)},\ \mathrm{side},\ p_{\mathrm{orig}},\ p,\ [i, j)), \tag{4}$$

so any claim in the report can be re-checked against the original bytes. When limits prevent retention, localisation degrades to a whole-file difference rather than to a guess.

E. Deterministic Policy

The policy maps a validated report to an exit code

$$c = \begin{cases} 2 & \text{input failure or incomplete analysis} \\ 3 & \text{complete, and } F \neq \emptyset \\ 0 & \text{complete, and } F = \emptyset \end{cases} \tag{5}$$

with code 1 reserved for unexpected internal errors. F is the set of default findings: `integrity-mismatch`, `identity-claim-conflict`, `lifecycle-source-absent` (an install hook in the tarball with no source counterpart) and `lifecycle-implicit-gyp`. Incomplete analysis takes precedence, and c does not depend on the model. Algorithm 1 summarises one check.

F. Advisory Model Layer

The adapter calls a fixed Google endpoint through an ordered route (policy `ordered-fallback-1`):

$$\begin{aligned} &\text{Gemini 3.5 Flash}^{\text{high}} \rightarrow \text{Gemini 3 Flash}^{\text{min}}_{\text{preview}} \\ &\quad \rightarrow \text{Gemma 4 26B}^{\text{min}}, \end{aligned} \tag{6}$$

where the superscript is the thinking level. Every tier runs at temperature zero with one candidate; Gemini tiers request JSON output with up to 4,096 output tokens, and the Gemma tier up to 1,000. Input is packed into a conservative envelope of 4,000 tokens. A request moves to the next tier only on a model-scoped HTTP 429, a transient 5xx or a timeout; an invalid output is retried on the same tier. There are at most three generation attempts, each capped at 45 s, within 135 s overall. The stage is disabled unless `UNMASK_MODEL=on` and quota limits are configured; offline CLI use additionally requires `--allow-model`, and requests are skipped when no

Algorithm 1 One UNMASK check

Require: exact spec or tarball T; optional source Z at SHA c; limits Λ

$b_T \leftarrow$ capture original bytes of T; record SHA-512
$A \leftarrow \textsc{ReadArchive}(b_T,\ \text{artifact},\ \Lambda)$
if Z available **then**
    $S \leftarrow \textsc{SelectPackage}(\textsc{ReadArchive}(Z,\ \text{source},\ \Lambda))$
else
    $S \leftarrow \emptyset$; $\kappa \leftarrow 0$
end if
for each regular $a \in A$ **do**
    assign $\sigma(a)$ by (2); localise if changed
end for
$K \leftarrow$ bounded syntax and manifest cues on A, S
$R \leftarrow \textsc{CreateReport}(A, S, \sigma, K)$; $\textsc{Validate}(R)$
$c \leftarrow \textsc{Policy}(R)$ ▷ fixed from here on
if model enabled **and** consent **and** eligible evidence **then**
    $H \leftarrow \textsc{ModelRoute}$(bounded evidence)
    keep $h \in H$ only if every quotation matches its evidence span
end if
return R, H, c

TABLE V
DEFAULT RESOURCE BUDGETS (SOURCE-LEVEL)

Area	Default
Compressed archive	50 MiB each
Expanded archive	250 MiB; ratio ≤ 100
Inventory	20,000 entries; 4,096 path bytes
Retained content	10 MiB/file; 128 MiB shared
Parse / fetch deadline	30 s each; redirects refused
JavaScript inspection	2 MiB/file; 200,000 tokens
Hosted worker	512 MiB V8 heap; 120 s
Queue	1 active + 1 waiting; 300 s TTL
Admission	6 checks / 10 min
Report store	200 entries; 64 MiB; 24 h TTL
Model	3 attempts; 45 s each, 135 s total; 4,000 in / 4,096 out

Source: `docs/stack.md`; model row from `src/model/client.mjs`.

evidence is eligible. A hypothesis h is accepted only if each quotation q it cites satisfies

$$q = \text{bytes}(\varepsilon)[k : k + |q|] \text{ for some } k, \tag{7}$$

for the evidence record ε it names. This establishes *where* the model looked, not whether its reasoning is correct; the report says so.

VII. Engineering and Evaluation Design

A. Resource Budgets

Table V lists the source-level defaults. They are budgets, not measured capacity, and the effective values are recorded in every report so that two runs can be compared only under the same limits.

TABLE VI
TEST DECLARATIONS ON MAIN (STATIC COUNT)

Suite	Files	Decl.	Covers
Product	22	330	Archive, compare, policy, report, model, server, runner, CLI, viewer
Tooling	11	58	Build, release policy, npm publication, smoke, toolchain
Total	**33**	**388**	

Source: `tests/`, counted with `grep` on top-level `test(` calls.

B. Continuous Integration

Every push and pull request runs one job, *Repository checks*, with `contents: read` permission and commit-pinned actions. It (1) installs the lockfile with `npm ci --ignore-scripts`; (2) checks that the running Node version, `.nvmrc`, package engines and lock engines agree and that direct dependencies are exact versions; (3) syntax-checks every tooling script; (4) checks Prettier formatting; (5) runs the test suite; (6) reports release readiness; and (7) inspects the npm archive allowlist. Product build and image smoke steps run only when readiness is true.

C. Release Integrity

The release design builds *one* npm tarball, tests the installed tarball, and builds the container image from those exact bytes. A release manifest binds version, Git tag, full source SHA, tarball name, size and SHA-256, and the GHCR image digest. Versions must be strictly greater than every existing stable version, compared numerically with `BigInt` segments; reruns of a used version are refused, and no step deletes or overwrites a published artifact. Release publication is now enabled for the 0.1.0 candidate, subject to these gates. The `Dockerfile` checks the SHA-256 of the packed tarball and the lockfile before installing, runs as a non-root user and labels the image with the release commit and tarball hash. The deployed service on Google Cloud Run (`asia-south1`) returns version `0.1.0`, commit `13f5504` and its tarball SHA-256 at `/version`.

D. Testing

Tests use the built-in `node:test` runner [15]. Table VI counts test declarations on the merged main branch; several are parameterised and expand into more cases at run time. `npm test` also installs the packed tarball into a temporary directory and invokes its real CLI binary.

E. Planned Evaluation Protocol

The analyzer now runs end to end, but no systematic accuracy benchmark has been run yet, so no accuracy figures are claimed. The evaluation that will gate Phase 2 is defined in advance:

- **Injection benchmark.** Take N popular npm releases with a resolvable `gitHead`; for each, create a modified tarball with one of k controlled edits (added file, changed byte, added `postinstall`). The detection rate is TPR =

TABLE VII
COMPONENT STATUS ON MAIN, 1 NOVEMBER 2026

Component	State	Evidence
Analyzer core	Merged	`src/core/`, 11 modules
HTTP server & queue	Merged	`src/server/`
CLI (hosted + offline)	Merged	`src/cli/`
Next.js website	Merged	`web/`, Next.js 16
Model route	Working	HTTP 200 from Gemini tiers
Tests	Merged	388 declarations, 33 files
Deployment	Live	Cloud Run, `/version` = 0.1.0
npm publication	Not yet	Registry returns 404
Accuracy benchmark	Not run	Protocol in Section VII

TP/(TP+FN) over modified tarballs, and the clean-release false-finding rate is FP/N.

- **Coverage.** Report the fraction of artifact bytes in each state, $\sum_{a:\sigma(a)=s} |a| \,/\, \sum_a |a|$, since a high not-compared share means the tool could not judge.
- **Cost.** Median and 95th-percentile wall time and peak RSS per check.
- **Model usefulness.** Citation-validity rate and a blinded human rating of whether each accepted hypothesis helps a reviewer.

VIII. CURRENT STATUS

Table VII reports the state of the upstream main branch on 1 November 2026. Most components that the first draft of this report listed as pending have since been merged.

Model status. The earlier HTTP 403 `PERMISSION_DENIED` on `gemma-4-26b-a4b-it` is resolved: the deployed service now reaches the Gemini tiers and receives HTTP 200 generations, and the team reports that the model works on its own checks.

Live spot checks. On 1 November 2026 we submitted three exact releases to the deployed service (Table VIII). Every deterministic analysis completed with full source coverage and exit code 0. For `ms` and `chalk` every shipped file was byte-identical to source, so the model stage was correctly skipped (`no-eligible-evidence`). For `clsx`, five shipped files had no source counterpart, so the evidence went to the model. The Gemini 3.5 Flash tier returned HTTP 200 in 37 s, but its output failed validation; a retry got a transient 503; the request then fell through to Gemini 3 Flash preview, which answered in 4 s but also failed validation. The report therefore showed the model as `unavailable` (`invalid-output`) and displayed no unchecked explanation. The deterministic result was unchanged, which is the intended behaviour. Three checks are a smoke test, not a measure of model quality.

Repository activity. Upstream main has 42 commits across eight merged pull requests from three contributors. This report adds a fourth contributor through its own pull request. Every pull request follows the repository's four-section template (what the PR does, models used, AI assistants, testing).

TABLE VIII
LIVE SPOT CHECKS ON THE DEPLOYED 0.1.0 SERVICE

Release	Ident.	Art.-only	Exit	Model stage
`clsx@2.1.1`	5	5	0	Called; output rejected
`ms@2.1.3`	4	0	0	Skipped, no evidence
`chalk@6.0.1`	12	0	0	Skipped, no evidence

Source: `/r/<id>.json` reports from the hosted service.

IX. DISCUSSION

A. Answers to the Design Questions

DQ1: Inert comparison. The working implementation shows that a useful comparison needs no execution: byte digests, conservative absence rules and byte spans are enough to separate identical, changed and extra files, and install hooks can be read from manifests. Whether this is useful *in practice* depends on how often packages ship generated code with no source counterpart; LastPyMile [11] suggests often, which is why not-compared and coverage are reported rather than hidden.

DQ2: Separated facts. Table II answers this directly. The key rule is asymmetric: weaker facts may *prioritise* review but never *clear* a stronger finding.

DQ3: Model role. The model can add value only as an explainer whose quotations are checked and whose output cannot change the exit code. The `clsx` check shows the guard working: two tiers produced output that failed validation, and the report showed nothing rather than an unverified claim. How often outputs pass validation, and whether accepted explanations help reviewers, is the next measurement.

B. Why Not the Alternatives

Rebuild and compare [7] gives stronger guarantees but executes untrusted build scripts and needs a faithful toolchain for every package; it is a natural extension once the inert core is proven. *Trust provenance alone* [6] covers only packages that publish it, and the current acquisition path does not verify signatures, so treating it as proof would overstate assurance. *AST or semantic diff* could explain minified changes but cannot establish equivalence in general and would enlarge the attack surface of the parser. *LLM as classifier* would be fast to demonstrate but would make the verdict depend on an unvalidated, non-deterministic component, contradicting DQ2.

C. Threats to Validity

Construct validity. Byte identity is a strict notion of correspondence; a benign rebuild that changes a timestamp is reported as changed. This is intended, but it will raise the share of findings a reviewer must inspect. **Internal validity.** Live results come from three packages on one deployed revision; model behaviour may vary between runs and tiers. **External validity.** Scope is npm only; PyPI and other ecosystems have different packaging conventions.

X. Limitations and Future Work

Each item states a measurable test of completion.

Publish to npm. *Done when* `@xagents_in/unmask@0.1.0` resolves on the public registry and its tarball SHA-256 matches `/version`.

Raise the model acceptance rate. Tune the evidence prompt and output schema. *Done when* the share of model calls that pass validation is measured over at least 50 packages with differences, and a human reviewer has rated the accepted explanations.

Run the injection benchmark. *Done when* TPR, false-finding rate, coverage and latency are reported for at least 100 releases.

Verify provenance. Implement Sigstore verification for npm provenance [3], [6]. *Done when* `provenance-verification-failed` is emitted by a real verifier.

Bind the deployment to a release. The service on Cloud Run already reports its commit and tarball hash. *Done when* those values match a published GitHub release manifest.

Extend scope. Add PyPI wheels and source distributions. *Done when* the injection benchmark runs on PyPI releases.

XI. Conclusion

UNMASK addresses a gap that registries leave open: the published package and the reviewed source are separate artifacts, and nothing checks that they agree. The project delivers a bounded, non-executing analyzer that keeps every fact at its true strength, ties every claim to bytes, and keeps the verdict independent of the language model. It is merged, covered by 388 test declarations, deployed on Cloud Run, and reachable through both a CLI and a Next.js website. Its model route now generates successfully, and its validator already refuses outputs it cannot verify. Publishing to npm, measuring the model's acceptance rate and running the evaluation benchmark are the work of the next phase, with completion criteria defined above.

Appendix A
Repository Structure and Reproduction

The main top-level parts are as follows. `src/` holds the analysis core, CLI, model adapter, server and viewer. `web/` holds the Next.js website. `docs/` holds the architecture, contracts, CLI, stack, release and website documentation. `scripts/` holds the build, package, toolchain, readiness, release-policy and candidate tooling. `tests/` holds product and tooling tests. The `Dockerfile` builds the serving image from the exact packed tarball. `.github/` holds the CI and release workflows, the `CODEOWNERS` file and the PR template. `report/` holds this report's LaTeX source. Reproduction requires Node.js 24.22.2 and the commands `npm ci --ignore-scripts`, `npm run build`, `npm test`, `npm run check` and `npm run check:package`.

Appendix B
Individual Contributions

Member	Primary responsibilities
Pranay Vishwakarma	*Project leadership; website shell (`src/viewer/shell.mjs`); Next.js website and report viewer (`web/`)*
Vaiditya Tanwar	*Architecture and contracts; analysis core and bounded server; three-tier model route; CI and release verification*
Vishesh Rao	*Package identity (`@xagents_in/unmask`); hosted CLI with explicit offline checks; 0.1.0 release configuration*
Sayan Bhattacharya	*Repository analysis and this IEEE-format project report*

Acknowledgment

The authors thank the organisers of Hacktoberfest 2026 Hack Day Sonipat and the faculty of Newton School of Technology, Rishihood University, for guidance, and the maintainers of Node.js, `tar-stream` and Acorn.

References

[1] npm, Inc., "Details about the event-stream incident," *The npm Blog*, Nov. 2018.

[2] NIST, "CVE-2024-3094: Malicious code in xz-utils upstream tarballs," National Vulnerability Database, 2024.

[3] Z. Newman, J. S. Meyers, and S. Torres-Arias, "Sigstore: Software signing for everybody," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2022, pp. 2353–2367.

[4] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, "in-toto: Providing farm-to-table guarantees for bits and bytes," in *Proc. 28th USENIX Security Symp.*, 2019, pp. 1393–1410.

[5] OpenSSF, "Supply-chain Levels for Software Artifacts (SLSA), specification v1.0," 2023. [Online]. Available: https://slsa.dev

[6] GitHub, "Generating provenance statements," *npm Docs*, 2023. [Online]. Available: https://docs.npmjs.com/generating-provenance-statements

[7] C. Lamb and S. Zacchiroli, "Reproducible builds: Increasing the integrity of software supply chains," *IEEE Software*, vol. 39, no. 2, pp. 62–70, 2022.

[8] M. Zimmermann, C.-A. Staicu, C. Tenny, and M. Pradel, "Small world with high risks: A study of security threats in the npm ecosystem," in *Proc. 28th USENIX Security Symp.*, 2019, pp. 995–1010.

[9] M. Ohm, H. Plate, A. Sykosch, and M. Meier, "Backstabber's knife collection: A review of open source software supply chain attacks," in *Proc. DIMVA*, LNCS 12223, 2020, pp. 23–43.

[10] R. Duan, O. Alrawi, R. P. Kasturi, R. Elder, B. Saltaformaggio, and W. Lee, "Towards measuring supply chain attacks on package managers for interpreted languages," in *Proc. NDSS*, 2021.

[11] D.-L. Vu, F. Massacci, I. Pashchenko, H. Plate, and A. Sabetta, "LastPyMile: Identifying the discrepancy between sources and packages," in *Proc. ESEC/FSE*, 2021, pp. 780–792.

[12] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[13] T. Gao, H. Yen, J. Yu, and D. Chen, "Enabling large language models to generate text with citations," in *Proc. EMNLP*, 2023, pp. 6465–6488.

[14] Gemma Team, Google DeepMind, "Gemma: Open models based on Gemini research and technology," arXiv preprint arXiv:2403.08295, 2024.

[15] OpenJS Foundation, "Node.js test runner (`node:test`)," Node.js v24 documentation, 2025.

[16] M. Buus, "tar-stream: A streaming tar parser and generator," v3.2.2. [Online]. Available: https://github.com/mafintosh/tar-stream

[17] M. Haverbeke *et al.*, "Acorn: A small, fast JavaScript-based JavaScript parser," v8.19.0. [Online]. Available: https://github.com/acornjs/acorn